Bert Collins

Dr. Wang

CPSC-224-01

10 October 2026

The program I created for HW-2 was a Java implementation of the board game Farkle. It was implemented with the intent of it being a singleplayer game, and the player being able to play multiple rounds of Farkle at a time. The program starts in Farkle.java, where an instance of the StartGame class is called, which starts the game. Everything else happens in the StartGame class file, where it uses another file, DiceWork, in order to manipulate, mix, and move the dice throughout the game. It is also responsible for printing out the final table. DiceWork is responsible for randomly shuffling the dice, changing the meld throughout the game, printing the table, rerolling the dice, detecting a hot hand, and computing the players score using the meld. I also added a TestCases file for the new assignment, which contains specific test cases in order to test functionality of the program. The tests can be enabled by toggling the test variable on or off (true or false) in the Farkle.java file. The design also consists of varying private and public class variables and functions, which is intended in a way to better organize the class files and its purposes. Public variables/functions are used outside the class, and private variables/functions are used only in the specific class. I also implemented a lot of helper functions throughout DiceWork, which aid other functions in fulfilling their purposes (such as computeScore). For this assignment, I also added unit tests in order to test specific dice rolls so I don’t have to waste time trying to get specific dice cases. One case that worked especially well was case 2, which was a farkle case. When selecting this test, you will instantly get a farkle upon the start of the game,

which proves that my farkle design works. Design issues I came across were implementing the hotHand function in DiceWork. This was very complicated because I wanted to keep the function clean and not just a bunch of if/else statements, as well as actually figuring out how to implement the true/false aspect of it, which had me stumped for a while. I ended up assigning each part of the dice a boolean factor, which starts off as false and becomes true when it “passes” a hotHand case. Once the entire array is true, then the function will return true, indicating a hot hand. Meanwhile for the logic, I just somewhat copied what computeScore did with the different scoring, and found myself copying and pasting much of the code that was already present there. However in the end, I realized that things I could’ve done differently if I had more time were not using as much for loops, and using less 2D arrays. The amount of for loops I used were a lot, and I realized that I could’ve made the program less redundant by cutting the amount of for loops I used. I also realized that the use of 2D arrays made sense in my head at the time, but later made my program more complicated than it should have been, which was also a pain to comment and explain later on.

```
bertcollins@Bert-Collins-Mac-5 BertCollinsHW1 % java Farkle.java
**************************************************************************
*                      Zag Farkle by Bert Collins!                       *
**************************************************************************
Please enter a player name:
Bert
Hey Bert! Welcome to Farkle!

Hand: 5 2 3 3 6 1
************************* Current hand and meld ******************
 Die   Hand |   Meld
------------+---------------
 ( A )    5   |
 ( B )    2   |
 ( C )    3   |
 ( D )    3   |
 ( E )    6   |
 ( F )    1   |
------------+---------------
 (K) BanK Meld & End Round
 (Q) Quit game
 (R) Reroll

 Score is: 0

 Bert, Enter letters of your choice:
```